

Data Security

Master's in Cybersecurity

João Marco Silva

Department of Informatics
joaomarco@di.uminho.pt

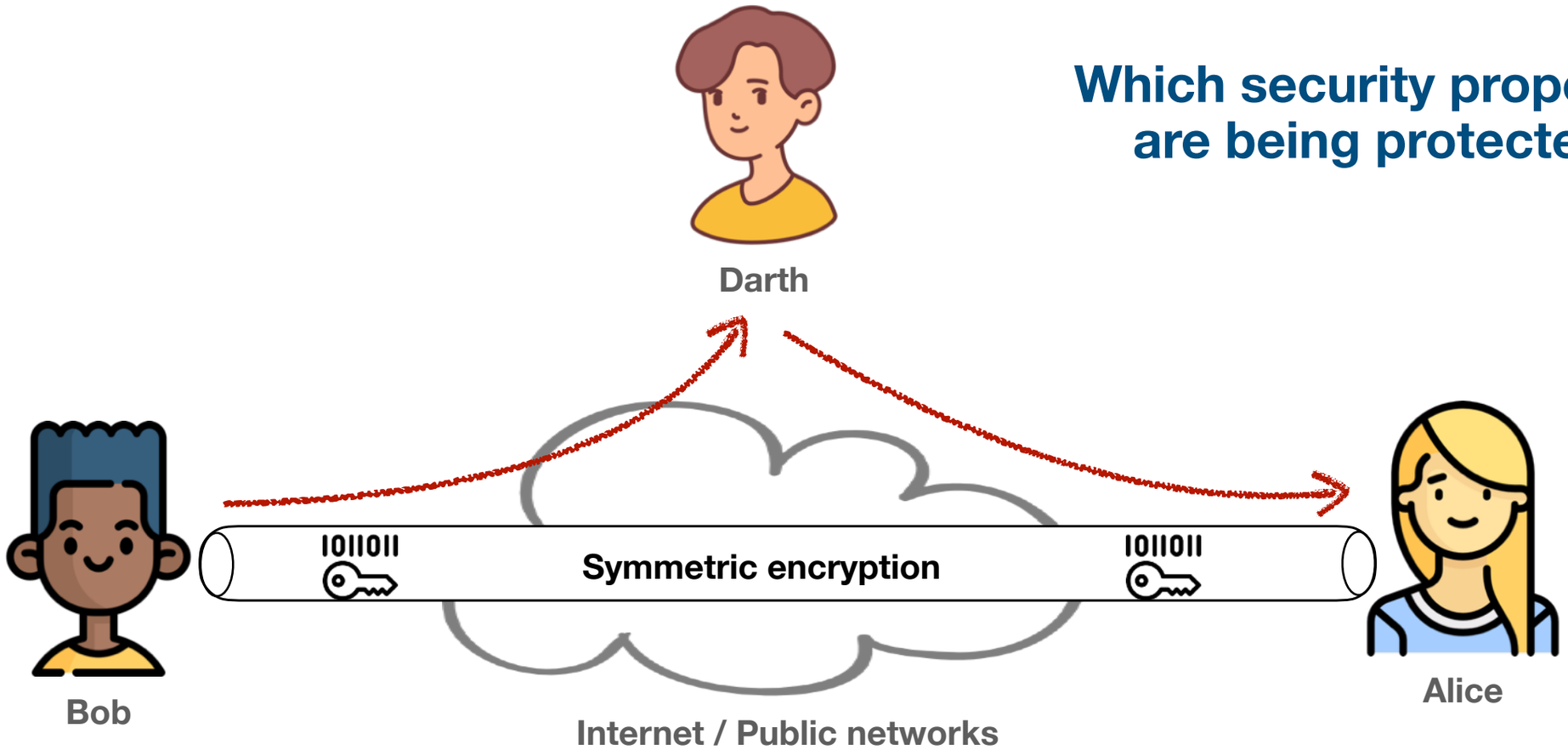
University of Minho
School of Engineering

Data Security

Objectives

- Understand the mechanisms used to ensure data integrity
- Comprehend the properties of cryptographic hash functions
- Understand the common attacks on hash functions
- Understand how to harden the use of cryptographic hash functions in practical applications

Recap



Which security properties are being protected?

- **Confidentiality**
- **Integrity**
- **Availability**
- **Authenticity**
- **Non-repudiation**

Data Integrity

Cryptographic Hash Functions

A ***hash function*** **`H`** accepts a variable-length block of data **`M`** as input and produces a fixed-size hash value **`h`.**

$$h = H(M)$$

Desired properties:

- The results of applying it to a large set of inputs will produce outputs that are evenly distributed and apparently random;
- The same message **`M`** hashed multiple times produces the same result **`h`**;
- A change to any bit in **`M`** results, with high probability, in a change to **`h`.**

Cryptographic Hash Functions

Hash functions for which is computationally infeasible to find:

- A data object that maps to the same hash result;
- Two data objects that map to the same hash result.

Hash functions are often used to determine whether data has changed. **<Integrity>**

Data Integrity

Cryptographic Hash Functions

Applications of Cryptographic Hash Functions

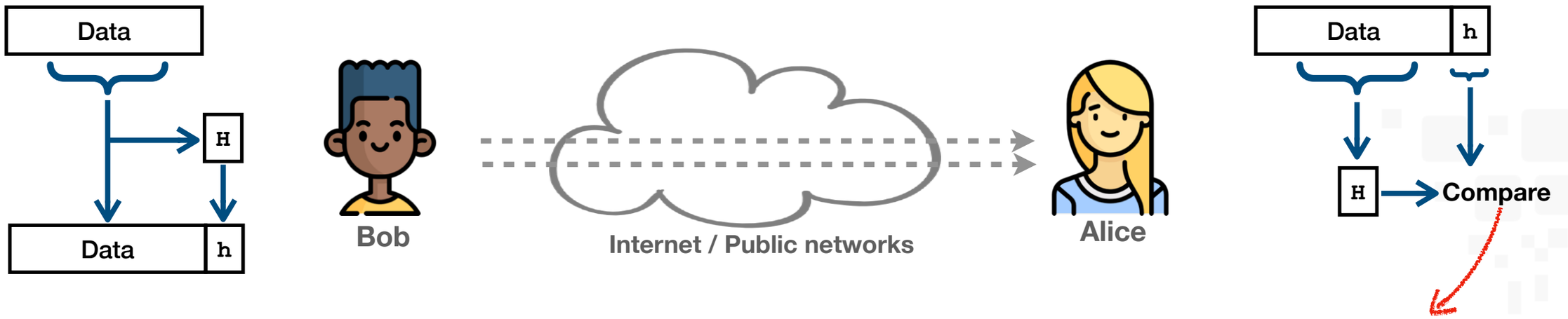
- Message Authentication
- Digital Signatures
- One-Way Password File
- Intrusion Detection
- Virus Detection
- Pseudorandom Functions (PRF)
- Pseudorandom Number Generator (PRNG)

Data Integrity

Cryptographic Hash Functions

A mechanism or service used to verify a message's integrity.

When used to ensure the sender's identity, the hash function value is also referred to as a *message digest*.

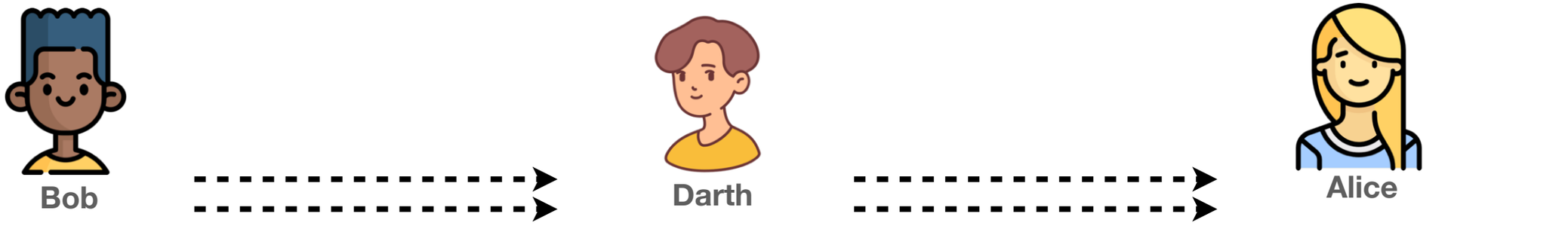


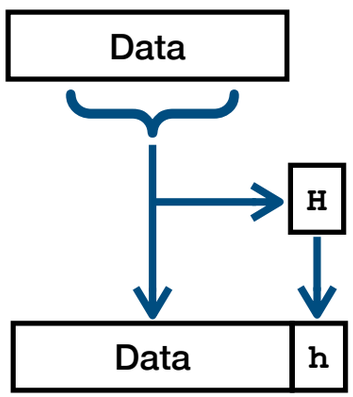
Data Integrity

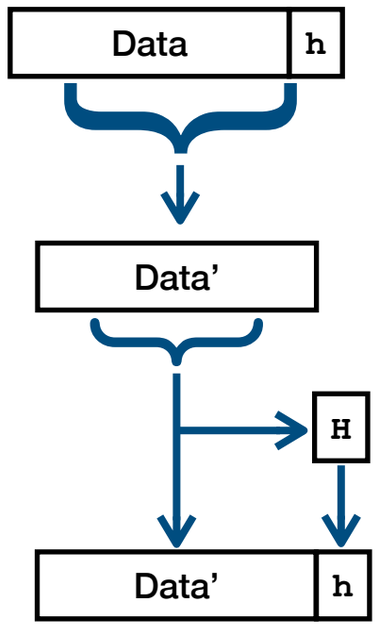
Cryptographic Hash Functions

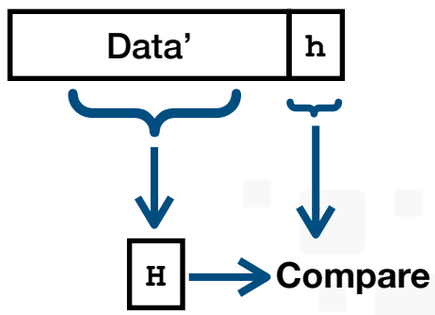
Attack against hash functions

The hash value must be transmitted securely.









Man-in-the-middle attack

Cryptographic Hash Functions

Requirements & Security

Terminology

For a hash value **`h = H(x)`**, **`x`** is called the **preimage** of **`h`**.

A **collision** occurs when **`x ≠ y`** and **`H(x) = H(y)`**.

- Since **`H`** is a many-to-one mapping, for any given **`h`**, there will be multiple possible preimages.
  - It maps a variable-length block (`n bits`) to a fixed-size value **`h`** (`b bits`). As typically `n > b`, on average, each `h` corresponds to 2^{n-b}.
- For data integrity, collisions are undesired.

Cryptographic Hash Functions

Requirements & Security

Security requirements for cryptographic use of hash functions

- **Variable input size:** `H` can be applied to a block of data of any size;
- **Fixed output size:** `H` produces a fixed-length output;
- **Efficiency:** `H(x)` is relatively easy to compute for any `x` (for software and hardware implementations);
- **Preimage resistant:** For any given hash value `h`, it is computationally infeasible to find `y` such that $H(y)=h$;
- **Second preimage resistant:** For any given block `x`, it is computationally infeasible to find $y \neq x$ with $H(y)=H(x)$;
- **Collision resistant:** It is computationally infeasible to find any pair `(x,y)` with $x \neq y$, such that $H(x)=H(y)$;
- **Pseudorandomness:** Output of `H` meets standard tests for randomness and unpredictability.

Cryptographic Hash Functions

Brute-Force Attacks

Preimage and Second Preimage Attacks

Objective

Find a value y such that $H(y)$ is equal to a given hash value h.

Brute-force method

Pick values of y at random and try each value until a collision occurs.

- For an m-bit hash value, the level of effort is proportional to 2^m;
- On average, an adversary would have to try 2^{m-1} values of y to find one that generates a given hash value h.

Cryptographic Hash Functions

Brute-Force Attacks

Collision resistant attacks / Birthday attack

Objective

Find two messages or data blocks, x and y, that yield the same hash function:

$$H(x)=H(y)$$

Brute-force method

It relies on the birthday paradox in probability theory:

In a group of 23 people, there is a 50% chance that at least two of them have the same birthday. Increasing the group to 50 people, the probability is over 97%.

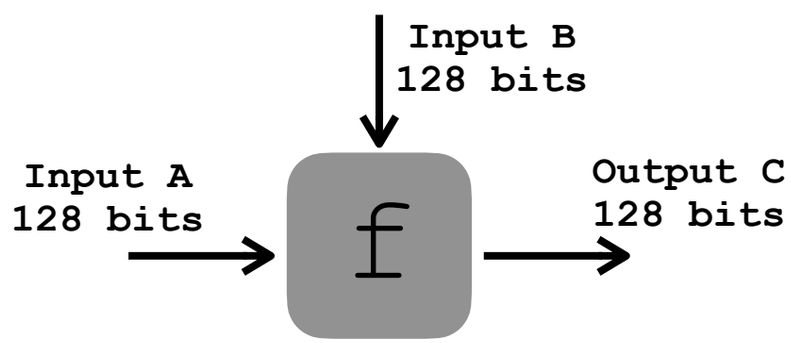
- When choosing random variables from a uniform distribution in the range 0 through $N - 1$, the probability that a repeated element is encountered exceeds 50% after $sqrt(N)$ choices have been made.
- Thus, for an m-bit hash value, by picking blocks randomly, it is expected to find two blocks with the same hash value within $2^{m/2}$ attempts.
  - For 128-bit and 512-bit hash functions, an attack should test 2^{64} and 2^{256} messages.

Cryptographic Hash Functions

Design

Compression function

A widely used component in secure hash function design. It consists of taking two inputs and producing an `n-bit` output.



They are one-way functions that:

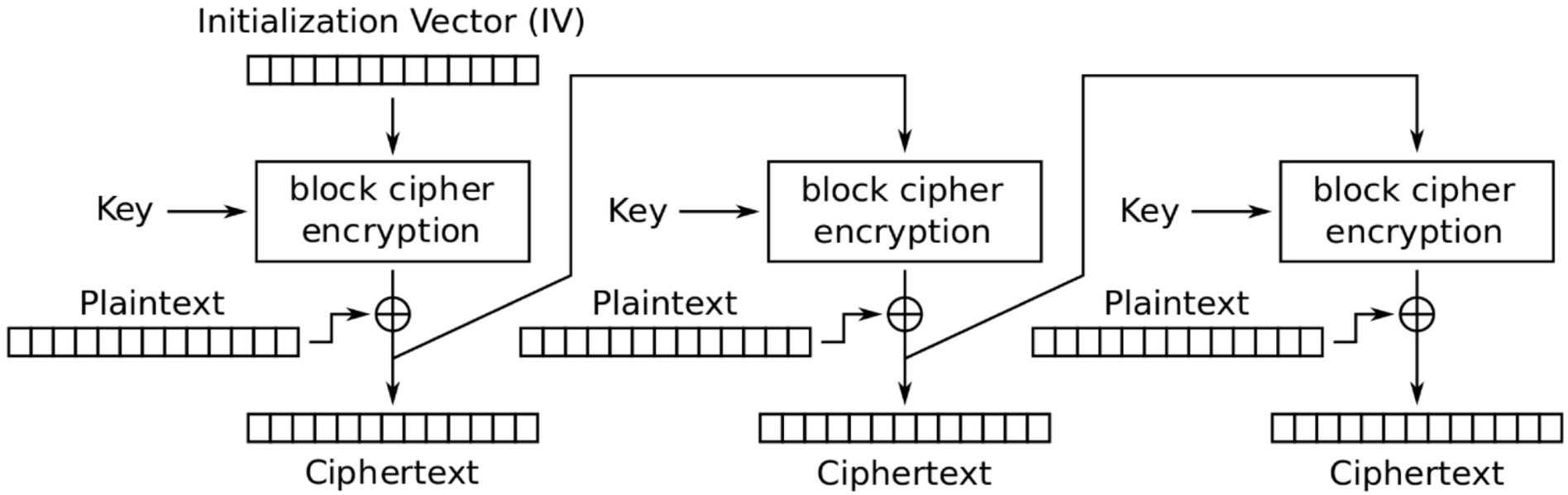
- Having the two inputs, it is easy to compute the output;
- It is infeasible to discover any of the inputs from the output;
- Having the output and one of the inputs, it is infeasible to find the second input;
- They are collision-resistant.

Cryptographic Hash Functions

Design

Compression function

Can we use chained block ciphers as hash functions?



Cipher Feedback (CFB)

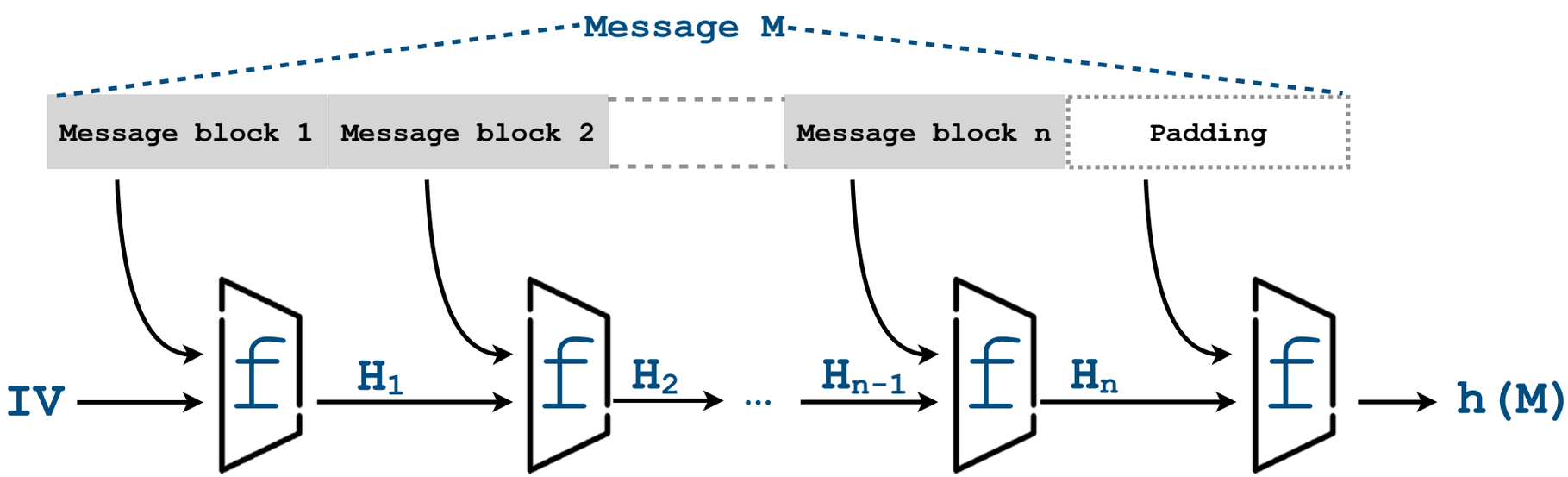
Only the last ciphertext could be used as a digest

Cryptographic Hash Functions

Design

Merkle-Damgård structure

Used by most of the current cryptographic hash functions, including **SHA**.



- The compression function chaining propagates the previous message hash;
- The `IV` is a fixed value defined by the algorithm;
- The padding should include information about the message size;
- If the compression function `f` is collision-resistant, so is the hash function.

Cryptographic Hash Functions

Standard schemes

Not recommended for applications with high-security requirements due to collisions already encountered.

See more here

Algorithm	Block size (bits)	Rounds	Digest size (bits)
MD5	512	4	128
SHA-1	512	4	160
SHA-224	512	64	224
SHA-256	1024	64	256
SHA-384		80	384
SHA-512			512
SHA3-224	1152	24	224
SHA3-256	1088		256
SHA3-384	832		384
SHA3-512	576		512

- SHA-2 Standard
- Considered secure for cryptographic application

- New standard from 2015;
- Uses Keccak's algorithm;
- Based on the sponge-construction with a wide range of configuration parameters that allow to adjust the level of security/efficiency.

Cryptographic Hash Functions

Salting

A technique that adds random data **S** to **M** before applying a hash function **H.**

$$h = H(M \,||\, S) \quad \text{or} \quad h = H(S \,||\, M)$$

Commonly used to store passwords

- Prevents identical hash values
  - Since it uses unique and pseudorandom **salt** values
- Prevents rainbow table attacks
- Mitigates Dictionary attacks
- Protects against brute-force attacks



Data Security

Master's in Cybersecurity

João Marco Silva

Department of Informatics
joaomarco@di.uminho.pt

University of Minho
School of Engineering